OCEANSENSE: AN IOT-BASED HYBRID SYSTEM FOR REAL-TIME DETECTION AND TRACKING OF MARINE DEBRIS

25-26J-002

DESIGN DOCUMENT (DD)

Rathnayake G R M S D – IT22228130

Supervisor: Ms. Dinithi Pandithage

BSc (Hons) in Information Technology Specializing in Computer Systems & Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

January 11, 2026

Table of Contents

1 Introduction

1.1 Purpose

The purpose of this document is to present the design specification of the Energy-Resilient Adaptive Communication (ERAC) protocol, developed as an individual contribution to the OceanSense project. This Design Document describes the architectural design, system interfaces, and key design considerations of ERAC as an energy-aware and resilient Medium Access Control (MAC) protocol for marine IoT networks.

The document provides sufficient technical detail to support system understanding and future implementation, while focusing exclusively on design structure and rationale rather than implementation results or experimental evaluation.

1.2 Scope

This document covers the design of the Energy-Resilient Adaptive Communication (ERAC) protocol as a system-level MAC-layer communication mechanism for point-to-point LoRa-based marine IoT nodes. The scope includes the system architectural design, external interfaces, energy-aware duty cycling, QoS-based packet scheduling, resilience mechanisms, and key performance and design constraints of the protocol.

The scope is limited to communication and energy-interaction aspects of the OceanSense system. Components such as debris detection, cloud analytics, swarm coordination and physical buoy design are outside the scope of this document and are addressed by other project contributors.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
ERAC	Energy-Resilient Adaptive Communication
MAC	Medium Access Control
IoT	Internet of Things
LoRa	Long Range wireless communication technology
QoS	Quality of Service
ENO	Energy Neutral Operation
EH	Energy Harvesting
SF	Spreading Factor
NS-3	Network Simulator 3

1.4 Overview

OceanSense is an IoT-based hybrid system for real-time detection and tracking of floating marine debris using a distributed network of autonomous sensor nodes deployed on the ocean surface. These nodes operate under dynamic marine conditions with intermittent connectivity and limited, energy-harvesting-based power availability.

Within this system, the Energy-Resilient Adaptive Communication (ERAC) protocol functions as a custom MAC-layer mechanism that enables energy-aware, priority-driven communication over point-to-point LoRa links. ERAC dynamically adapts duty cycles and transmission parameters based on predicted energy availability while prioritizing latency-critical debris alerts over routine telemetry.

This document presents a structured design view of ERAC, describing its role, architecture, and key design considerations to support subsequent implementation and validation phases.

2 Overall Description

2.1 Product Perspective

The Energy-Resilient Adaptive Communication (ERAC) protocol is designed as a subsystem-level MAC-layer component within the OceanSense marine IoT architecture. It interfaces directly with the LoRa physical layer, the energy management subsystem, and the application layer responsible for sensing and event generation.

ERAC is not a standalone product; it functions as an integrated protocol that enhances the reliability, energy efficiency, and resilience of the overall system. The protocol replaces generic LoRaWAN-style random access mechanisms with a custom, adaptive MAC optimized for energy-harvesting marine sensor nodes.

Positioned between the application layer and the LoRa physical layer, ERAC continuously adapts its behavior based on network conditions and energy availability, enabling long-term autonomous operation in dynamic marine environments.

2.1.1 System Interfaces

ERAC interfaces with the underlying embedded operating environment and simulation platforms through standard operating system and firmware services. These interfaces support task scheduling, timing control, power management, and memory handling required for protocol operation on low-power devices.

2.1.2 User Interfaces

ERAC does not provide a direct graphical user interface. As a MAC-layer protocol, it operates autonomously within embedded sensor nodes and is transparent to end users. Limited interaction is provided through configuration parameters and diagnostic outputs used for development, monitoring, and evaluation purposes.

2.1.3 Hardware Interfaces

ERAC interfaces with embedded hardware components including a microcontroller, LoRa transceiver, energy storage system, energy harvesting sources, and an external processing unit (e.g., Raspberry Pi). Communication between the microcontroller and the external processor is supported via a serial interface.

2.1.4 Software Interfaces

ERAC interfaces with the application layer, energy management software, and supporting firmware services. In addition, the protocol is integrated with the NS-3 simulation framework for design validation and performance evaluation. ERAC also interfaces with higher-level software modules running on an external processor for data forwarding and system coordination.

2.1.5 Communication Interfaces

ERAC uses direct point-to-point LoRa communication between sensor nodes and a coordinator node. This communication model enables flexible control over timing, scheduling, and duty cycle without reliance on full LoRaWAN infrastructure.

2.1.6 Memory Constraints

ERAC is designed to operate within the limited memory resources of low-power embedded platforms. Memory usage is constrained to small buffers and state variables required for protocol operation.

2.1.7 Operations

ERAC operates autonomously by monitoring energy conditions, scheduling transmissions, and adapting communication parameters. The protocol supports both normal periodic operation and event-driven behavior under dynamic network conditions.

2.1.8 Site Adaptation Requirements

ERAC is intended for deployment in near-surface marine environments. Minor configuration adjustments may be required based on environmental conditions and regulatory constraints, while core protocol logic remains unchanged.

2.2 Product Functions

At a high level, ERAC provides the following key functions:

- Adaptive control of MAC-layer channel access
- Predictive energy-aware duty cycling
- Priority-based scheduling of outgoing packets
- Dynamic adjustment of LoRa transmission parameters
- Resilient communication under intermittent connectivity

These functions collectively enable reliable and energy-neutral communication for marine debris monitoring.

2.3 User Characteristics

Primary users of ERAC are system developers, researchers, and network engineers involved in marine IoT deployments. These users are expected to have:

- basic knowledge of wireless communication protocols,
- familiarity with embedded systems or network simulation tools, and
- understanding of energy-constrained IoT environments.

2.4 Constraints

The design of ERAC is subject to the following constraints:

- limited energy availability from stochastic harvesting sources,
- regulatory limits on transmission power and duty cycle,
- restricted memory and processing capabilities of embedded devices,
- variable and unpredictable marine communication conditions.

2.5 Assumptions and Dependencies

The design assumes:

- availability of solar and/or wave energy harvesting,
- LoRa-compatible transceiver hardware,
- stable basic firmware operation on the MCU,
- availability of NS-3 for simulation-based validation.

ERAC depends on accurate energy measurements and reliable timer services to function correctly.

2.6 Apportioning of Requirements

Requirements defined in this document are prioritized as follows:

1. Energy neutrality and system survivability
2. Reliable delivery of high-priority debris alerts
3. Efficient handling of routine telemetry traffic
4. Scalability and resilience under network dynamics

This prioritization guides both design decisions and future implementation phases.

3 Specific Requirements

This section defines the design-level requirements of the Energy-Resilient Adaptive Communication (ERAC) protocol. It specifies the external interfaces, architectural design, performance expectations, and system attributes required to implement ERAC on energy-harvesting marine IoT nodes.

3.1 External Interface Requirements

3.1.1 User Interfaces

ERAC does not expose a direct graphical user interface. As a MAC-layer protocol, it operates autonomously within the microcontroller firmware.

Indirect interaction is supported through:

- configurable protocol parameters (e.g., duty-cycle limits, priority levels),
- diagnostic and debug outputs during development and testing, and
- status information accessible via serial logs.

These interfaces are intended for developers and system evaluators rather than end users.

3.1.2 Hardware Interfaces

ERAC interfaces with the following hardware components:

- **Microcontroller Unit (MCU)**: Executes ERAC logic, manages timing events, and controls low-power sleep states.
- **LoRa Transceiver:** Provides wireless communication with configurable parameters such as Spreading Factor and transmission power.
- **Energy Storage System:** Supplies battery voltage and state-of-charge measurements used for energy-aware decision making.
- **Energy Harvesting Modules:** Solar and wave energy sources providing stochastic power input to the system.
- **Peripheral Subsystems (Camera, Sensors, Actuators, External Processor etc.):** High-power components that contribute to overall node energy consumption but are not directly controlled by ERAC.
- **Timers and Interrupts:** Support duty-cycle control, backoff scheduling, and periodic energy evaluation.

ERAC operates within the processing, memory, and power constraints of low-power embedded platforms while adapting communication behavior based on system-level energy availability.

3.1.3 Software Interfaces

ERAC interfaces with the following software components:

- **Application Layer:** Provides data packets and assigns priority levels based on event type.
- **Energy Management Module:** Supplies real-time energy measurements and short-term energy predictions.
- **Operating System / Firmware Services:** Provide task scheduling, timer management, and power control.
- **Network Simulation Framework (NS-3):** Used for simulation-based validation of protocol behavior and performance.

These interfaces enable cross-layer interaction while preserving modularity and portability.

3.1.4 Communication Interfaces

ERAC supports two primary communication interfaces:

1. **Node-to-Gateway Communication:**
   Wireless communication between sensor nodes and the coordinator node is performed using direct point-to-point LoRa links. ERAC controls channel access, duty cycle, packet prioritization, and transmission parameters over this interface.

2. **MCU-to-External Processor Communication:**
   Communication between the microcontroller executing ERAC and an external processing unit (e.g., Raspberry Pi) is performed via a UART-based serial interface. This interface is used to transfer validated sensor data, event notifications, and protocol status information from the MCU to the external processor for aggregation and uplink forwarding.

3.2 Architectural Design

3.2.1 High-Level Architectural Design

ERAC is implemented as a **cross-layer MAC protocol** embedded within the microcontroller firmware of each sensor node.

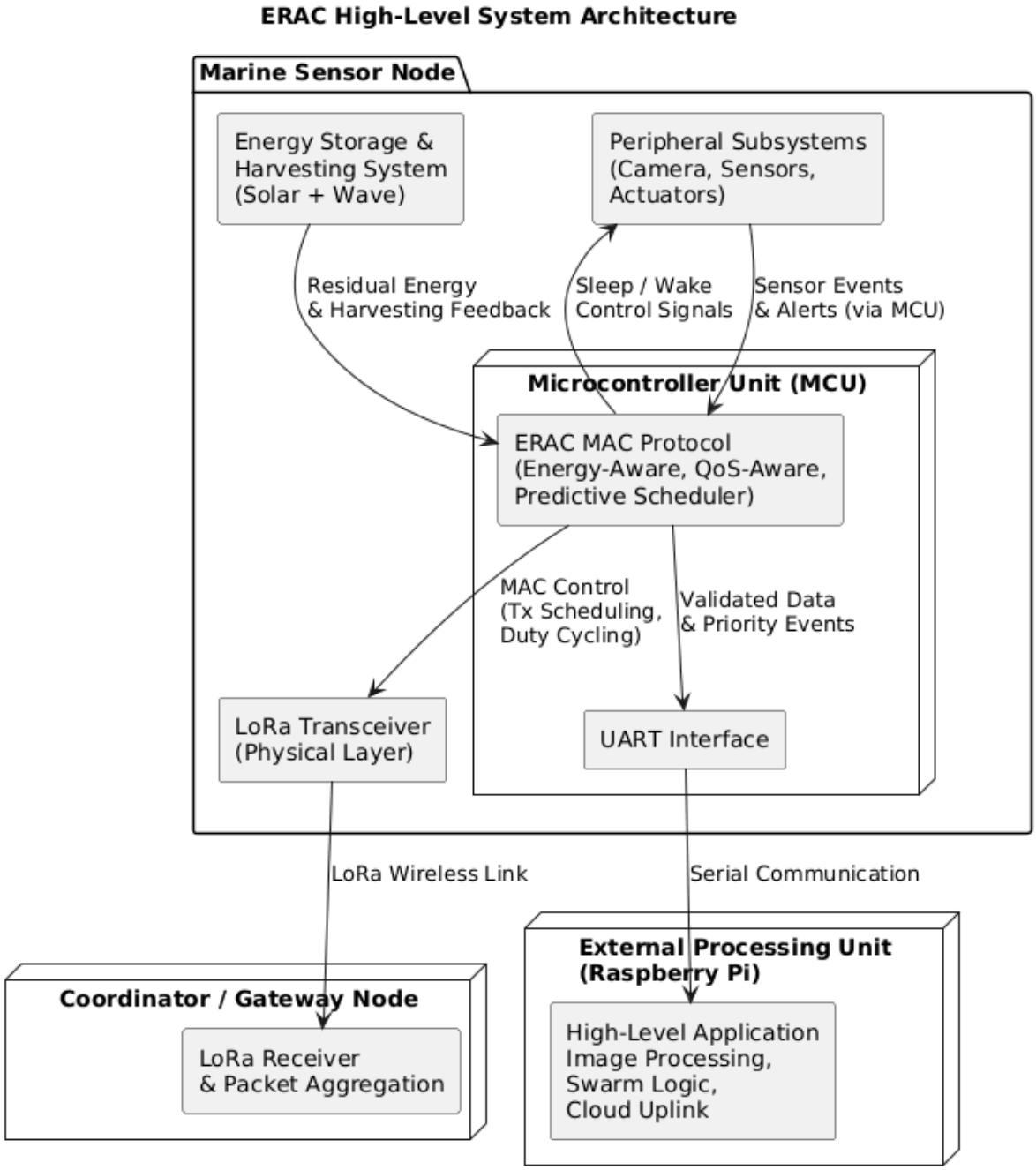

Figure 1 - High-Level Architectural Design

These components operate cooperatively to balance energy neutrality, reliability, and latency.

3.2.2 Hardware and Software Requirements with Justification

Hardware Requirements

- **Microcontroller Unit (MCU)**
  Requirement: Low-power MCU supporting deep sleep modes, hardware timers, UART, and SPI interfaces.
  Justification: Required to execute ERAC MAC logic, enforce duty cycling, and interface with the LoRa transceiver efficiently.
  Example platforms: ESP32-class or STM32-class microcontrollers.

- **LoRa Transceiver Module**
  Requirement: Sub-GHz LoRa transceiver supporting configurable Spreading Factor, bandwidth, and transmission power.
  Justification: Enables adaptive link control under varying channel and energy conditions.
  Example platforms: SX127x / SX126x series transceivers.

- **Energy Harvesting Sources**
  Requirement: Solar and/or wave energy harvesting modules capable of supplying intermittent low-power input.
  Justification: Required to support long-term autonomous and energy-neutral operation in marine environments.
- **Energy Storage and Monitoring Circuitry**
  Requirement: Rechargeable battery system with voltage and current sensing capability.
  Justification: Provides system-level energy state information used by ERAC for energy-aware MAC decisions.

3. **Peripheral Subsystems**
   Requirement: High-power sensing and actuation modules interfaced via standard digital interfaces.
   Justification: These subsystems contribute significantly to total node energy consumption; ERAC accounts for their impact indirectly through system-level energy monitoring rather than direct control.

4. **External Processing Unit**
   Requirement: Embedded Linux–based processor supporting serial communication with the MCU.
   Justification: Handles computationally intensive tasks such as image processing and uplink forwarding, allowing ERAC to remain lightweight and energy-efficient on the MCU.
   Example platforms: Raspberry Pi–class single-board computers.

Software Requirements

- **Embedded Firmware Environment**
  Requirement: Firmware supporting hardware timers, interrupts, UART communication, and power management APIs.
  Justification: Required for precise scheduling, backoff control, and duty-cycle enforcement by ERAC.

- **Energy Estimation and Prediction Algorithms**
  Requirement: Lightweight algorithms capable of estimating short-term energy availability.
  Justification: Enable proactive adaptation of MAC behavior based on expected energy conditions.

5. **Higher-Level Application Software**
   Requirement: Software modules running on the external processor for sensor processing, data aggregation, and uplink communication.
   Justification: Separates energy-critical MAC operations from compute-intensive application tasks.

- **Simulation environment (NS-3)**
  Requirement: Discrete-event network simulator with energy modeling support.
  Justification: Allows scalable and repeatable validation of protocol behavior.

3.2.3 Risk Mitigation Plan with Alternative Solution Identification

Risk	Impact	Mitigation Strategy	Alternative
Energy depletion	Node shutdown	Predictive duty cycling	Conservative fixed duty cycle
Packet collisions	Data loss	Priority-based backoff	Randomized ALOHA
Link occlusion	Connectivity loss	Adaptive SF adjustment	Fixed SF configuration
Energy prediction error	Inefficient scheduling	Short-term correction	Threshold-based control
MCU overload	Timing violations	Offload processing to Pi	Simplified MAC logic
Incorrect packet priority assignment	Critical data delayed	Priority classification at application layer	Static priority mapping

The design prioritizes graceful degradation over abrupt failure.

3.2.4 Cost–Benefit Analysis for the Proposed Solution

Costs

- Slight increase in firmware complexity
- Additional computation for prediction and scheduling

Benefits

- Extended network lifetime through energy neutrality
- Improved delivery of critical event data
- Reduced packet collisions and retransmissions
- Greater resilience under dynamic marine conditions

The benefits significantly outweigh the costs for long-term autonomous deployments.

3.3 Performance Requirements

ERAC is designed to meet the following performance requirements:

- Low-latency delivery of high-priority packets under event-driven traffic conditions
- Packet delivery ratio exceeding baseline LoRa MAC approaches in dynamic marine environments
- Energy-neutral operation over extended deployment periods
- Bounded memory and processing overhead suitable for low-power microcontrollers

Performance requirements are evaluated while coexisting with other node subsystems, including sensing peripherals and external processing units, without assuming exclusive access to system energy resources.

3.4 Design Constraints

The design of ERAC is constrained by:

- limited and stochastic energy availability from harvesting sources,
- regulatory transmission limits on duty cycle and power,
- constrained memory and processing capacity of embedded devices,
- dynamic and unpredictable marine communication conditions, and
- shared energy usage with sensing and processing subsystems such as cameras and external processors.

These constraints shape all architectural and scheduling decisions.

3.5 Software System Attributes

3.5.1 Reliability

ERAC improves reliability through adaptive link control, priority scheduling, and congestion-aware backoff mechanisms.

3.5.2 Availability

The protocol ensures high availability by maintaining energy neutrality and preventing premature node shutdown.

3.5.3 Security

Basic security is ensured by limiting transmission exposure and avoiding unnecessary retransmissions. Advanced security mechanisms are outside the scope of this document.

3.5.4 Maintainability

ERAC is modular in design, allowing individual components such as energy prediction or scheduling logic to be updated independently.

3.6 Other Requirements

Additional requirements include:

- compatibility with simulation and hardware environments,
- support for incremental enhancement,
- extensibility to future marine IoT applications.

4 Appendices

Appendix A – ERAC MAC Scheduling Algorithm (Pseudocode)

```
Input:
  PacketQueue Q
  ResidualEnergy E_res
  PredictedEnergy E_pred
  EnergyThreshold E_th
  HeartbeatInterval T_hb
  CurrentTime t

While scheduler is active:

  If Q is empty:
    Enter sleep state
    Exit scheduler loop

  pkt ← dequeue(Q)

  RequiredEnergy ← TxBaseEnergy + TxEnergyPerByte × pkt.payloadSize

  If E_res < RequiredEnergy:
    Re-enqueue(pkt)
    Defer transmission
    Continue next round

  Update ERAC operational state
  Update energy prediction using EWMA

  If pkt.priority == HIGH:
    Transmit(pkt)

  Else if pkt.priority == MEDIUM:
    If E_pred ≥ E_th:
      Transmit(pkt)
    Else:
      Re-enqueue(pkt)

  Else if pkt.priority == LOW:
    If (t – lastHeartbeatTime) ≥ T_hb:
      Transmit(pkt)        // Heartbeat exception
      lastHeartbeatTime ← t
    Else if E_pred ≥ (E_th + margin):
      Transmit(pkt)
    Else:
      Re-enqueue(pkt)

  Schedule next scheduler round
```

Appendix B - Simulation Validation Snapshot

Purpose

This appendix provides minimal validation evidence showing that the ERAC MAC design behaves as intended under simulation.

Simulation Setup (Summary)

- Simulator: ns-3
- MAC Scheduler Interval: 1 second
- Energy Sources: Solar + Wave
- Traffic Types: HIGH, MEDIUM, LOW (heartbeat)

Sample Output (CSV Extract)

```
Time,ResidualEnergy,PredictedEnergy,Solar,Wave,TxEnergy,Priority,State
0,3.00,2.83,0.00,0.00,0.00,LOW,NORMAL
1,3.24,3.54,0.40,0.24,0.40,MEDIUM,NORMAL
2,3.55,3.89,0.40,0.18,0.28,HIGH,NORMAL
3,3.86,4.18,0.40,0.15,0.24,LOW,NORMAL
```

Observed Behavior

- **HIGH priority packets** transmit immediately.
- **MEDIUM packets** depend on predicted energy.
- **LOW (heartbeat) packets** respect interval constraints.
- Predicted energy tracks harvesting trends smoothly.

Conclusion

The simulation results confirm that:

- Energy prediction influences MAC behavior,
- QoS priorities are respected,
- The protocol adapts to changing energy conditions.

Appendix C – Implementation Evidence

1. Implementation Evidence

```
sdr@SDR:~/ns-3.46.1$ ./ns3 run "scratch/erac-mac-v3 --scenario=mixed --simTime=20"
```

2. Scheduler Output Snapshot

```
⚡ Energy Harvesting | Solar: 0.4 J | Wave: 0.114478 J | SoC: 4.65016 J

╔═══════════════════════════════════╗
  ERAC MAC SCHEDULER – Round 4
╚═══════════════════════════════════╝

Residual Energy  : 4.65016 J
Predicted Energy : 4.01541 J
State: NORMAL | Alpha: 0.6
📊 Predicted Energy (next step): 4.53695 J
🔴 HIGH priority → Immediate transmission
✅ Transmission successful | TX Energy: 0.28 J
⚡ Energy Harvesting | Solar: 0.4 J | Wave: 0.110722 J | SoC: 4.88089 J

╔═══════════════════════════════════╗
  ERAC MAC SCHEDULER – Round 5
╚═══════════════════════════════════╝

Residual Energy  : 4.88089 J
Predicted Energy : 4.53695 J
State: NORMAL | Alpha: 0.6
📊 Predicted Energy (next step): 4.88174 J
🟡 MEDIUM priority → Sufficient predicted energy → TX
✅ Transmission successful | TX Energy: 0.4 J
⚡ Energy Harvesting | Solar: 0 J | Wave: 0.241983 J | SoC: 4.72287 J

╔═══════════════════════════════════╗
  ERAC MAC SCHEDULER – Round 6
```

3. Energy Log Output (CSV)

erac-mac-v3.cc | erac-scenarios.h | erac_energy_log.csv

scratch > erac_energy_log.csv > data

```
Time,ResidualEnergy,PredictedEnergy,Solar,Wave,TxEnergy,Priority,State
0,2.76,2.832,0,0,0.24,LOW,NORMAL
2,4.13569,4.01541,0.4,0.148672,0.4,MEDIUM,NORMAL
3,4.37016,4.53695,0.4,0.114478,0.28,HIGH,NORMAL
4,4.48089,4.88174,0.4,0.110722,0.4,MEDIUM,NORMAL
5,4.48287,4.82268,0,0.241983,0.24,LOW,SAVING
6,4.11313,4.65489,0,0.0302658,0.4,MEDIUM,SAVING
7,4.04857,4.51363,0,0.175439,0.24,LOW,SAVING
8,3.93028,4.45914,0,0.281712,0.4,MEDIUM,SAVING
9,3.87812,4.32918,0,0.187831,0.24,LOW,SAVING
10,3.7247,4.25781,0,0.246585,0.4,MEDIUM,SAVING
12,3.56619,4.10844,0,0.0737052,0.4,MEDIUM,SAVING
14,3.42316,3.94065,0,0.00597561,0.4,MEDIUM,SAVING
16,3.55795,3.9468,0,0.283196,0.4,MEDIUM,SAVING
18,3.42706,3.83243,0,0.0185096,0.4,MEDIUM,SAVING
```

Observed Behavior

- **HIGH priority packets** transmit immediately.
- **MEDIUM packets** depend on predicted energy.
- **LOW (heartbeat) packets** respect interval constraints.
- Predicted energy tracks harvesting trends smoothly

Conclusion

The simulation results confirm that:

- Energy prediction influences MAC behavior,
- QoS priorities are respected,
- The protocol adapts to changing energy conditions.

5 References

[1] A. Kansal et al., “Power management in energy harvesting sensor networks,” ACM Transactions on Embedded Computing Systems.

[2] F. Adelantado et al., “Understanding the limits of LoRaWAN,” IEEE Communications Magazine.

[3] J. Polastre et al., “Versatile low power media access for wireless sensor networks,” ACM SenSys.